

Project 2: SnailSaver 多核性能优化与碰撞检测 Write-up

一、耗时记录表

设计	编码	调试	测试	会议
1h	2h	5h	2h	10h

二、整体架构演进

原始代码在遍历过程中直接进行 `intersect` 判断并用链表生成碰撞事件，导致数据收集与计算高度耦合。为了突破性能瓶颈并为后续的并行化铺平道路，我对整体架构进行了重构，拆分为职责分明的两个阶段：

- Broad Phase**: 引入四叉树，仅负责空间划分并收集所有可能发生碰撞的“候选线段对”，存入 `CandidatePairBuffer` 并去重。该阶段不夹杂任何复杂相交几何计算，职责单一。
- Narrow Phase**: 对候选对数组进行统一的相交检测。结果存入动态数组 `IntersectionEventBuffer`，经排序后依次进行物理求解。

这种解耦不仅便于后续的并行化改造，更通过排序机制严格保证了确定性的求解顺序。

三、Part 1: 四叉树 Broad Phase 的深入优化

3.1 四叉树的构建与边界处理

在 `quadtree.c` 的 `insert_line` 中，我将四叉树节点容量设为 `MAX_LINES = 16`，最大深度设为 `MAX_DEPTH = 8`。

- 跨越边界处理**: 插入线段时，若线段能被子节点完全包含，则下沉；若跨越了多个子节点的边界，则直接留在当前父节点。
- 节点分裂**: 当无子节点且节点内线段数满载、且未达最大深度时，调用 `Quadtree_split` 创建四个子节点，并将父节点原有的线段重新分配下沉。
- 包围盒修正**: 原代码使用 `0.5 * BOX_XMAX` 作为中心，我将其优化为基于 `fmax(BOX_XMAX - BOX_XMIN, BOX_YMAX - BOX_YMIN)` 的正方形包围盒，确保无论输入窗口长宽比如何，四叉树都能完美覆盖模拟区域。

3.2 候选对的无冗余收集

在 `collect_candidate_pairs` 函数中，采用了递归遍历与向上追溯结合的策略：

- 节点内部**: 枚举当前节点内部的所有线段对。
- 跨层级配对**: 由于跨界线段停留在父节点，设计了 `Quadtree_collectAgainstAncestors` 函数。深层节点会主动向上遍历 `parent` 链，将自身的线段与所有祖先节点的线段配对。单向向上追溯的设计避免了浅层向下比对带来的重复检测。

3.3 排序与去重机制

因为一条线段的某些部分可能被子节点检测，同时自身又挂在父节点，仍可能产生重复候选对。对此，我将收集到的对按 `(l1.id, l2.id)` 进行快排，随后线性扫描去除相邻重复项，最终输出绝对无冗余的有序候选对数组。

四、Part 2：OpenCilk 并行化与竞争消除策略

4.1 动态并行阈值判定

为了避免在微小工作量下因线程调度开销导致性能倒退，设计了 `CollisionWorld_shouldRunParallel` 拦截器。仅当 `__cilkrts_get_nworkers() > 1` 且计算任务量 `workItems ≥ 512` 时，才激活并行模式。

4.2 位置更新与墙壁碰撞的无锁并行

- 位置更新 (`updatePosition`)：每条线段根据自身速度更新坐标，绝对独立，直接使用 `cilk_for` 展开。
- 墙壁碰撞 (`lineWallCollision`)：墙壁碰撞需要累加全局碰撞计数器 `numLineWallCollisions`。若直接多线程累加会导致数据竞争。为此，开辟了长度等于线段数的局部标记数组 `unsigned char *collided`。在并行循环中，各线程只修改对应的 `collided[i]` 标志位和自身速度；并行结束后，再串行扫一遍数组汇总总碰撞数。这种方法巧妙且彻底地避免了加锁开销。

4.3 窄相检测并行与核心 Reducer 设计

这是整个并行优化的核心精妙之处。

窄相阶段需并发调用 `intersect`，多个 Worker 会同时向 `IntersectionEventBuffer` 写入事件。为了实现无锁并发，我利用了 Cilk 的 Reducer 机制：

- 定义 Reducer：基于自建的动态数组，定义了 `IntersectionEventBuffer_identity` 和 `IntersectionEventBuffer_reduce`。
- 工作流：在 `cilk_for` 循环体内，每次 `append` 都在 Worker 自身的局部视图（通过 `identity` 初始化为空）中进行，互不干扰。循环结束时，Cilk 运行时自动调用 `reduce`，将右侧视图的数据拷贝/移动追加到左侧末尾并释放右侧内存。

通过这一机制，实现了极致高效的无锁并行事件收集。收集完毕后，对最终的 Buffer 进行快速排序，再安全地交给串行求解器执行，严格保证了物理模拟的准确性与一致性。

不同CILK_NWORKERS下测试结果

```
1 CILK_NWORKERS=2
2 Number of frames = 4000
3 Input file path is: input/mit.in
4 ---- RESULTS ----
5 Elapsed execution time: 2.158117s
6 1262 Line-Wall Collisions
7 19806 Line-Line Collisions
8 ---- END RESULTS ----
9 CILK_NWORKERS=4
10 Number of frames = 4000
```

```
11 Input file path is: input/mit.in
12 ---- RESULTS ----
13 Elapsed execution time: 2.059187s
14 1262 Line-Wall Collisions
15 19806 Line-Line Collisions
16 ---- END RESULTS ----
17 CILK_NWORKERS=8
18 Number of frames = 4000
19 Input file path is: input/mit.in
20 ---- RESULTS ----
21 Elapsed execution time: 1.718578s
22 1262 Line-Wall Collisions
23 19806 Line-Line Collisions
24 ---- END RESULTS ----
25 CILK_NWORKERS=16
26 Number of frames = 4000
27 Input file path is: input/mit.in
28 ---- RESULTS ----
29 Elapsed execution time: 1.771711s
30 1262 Line-Wall Collisions
31 19806 Line-Line Collisions
32 ---- END RESULTS ----
33 CILK_NWORKERS=24
34 Number of frames = 4000
35 Input file path is: input/mit.in
36 ---- RESULTS ----
37 Elapsed execution time: 1.853017s
38 1262 Line-Wall Collisions
39 19806 Line-Line Collisions
40 ---- END RESULTS ----
41 CILK_NWORKERS=32
42 Number of frames = 4000
43 Input file path is: input/mit.in
44 ---- RESULTS ----
45 Elapsed execution time: 1.937912s
46 1262 Line-Wall Collisions
47 19806 Line-Line Collisions
48 ---- END RESULTS ----
```

CILK_NWORKERS	执行时间 (s)
2	2.158117
4	2.059187
8	1.718578
16	1.771711
24	1.853017
32	1.937912

在CILK_NWORKERS = 8时取得最佳性能。

不同depth与MAX_DEPTH下的测试结果

```
1 depth=1
2 Number of frames = 4000
3 Input file path is: input/mit.in
4 ---- RESULTS ----
5 Elapsed execution time: 2.586612s
6 1262 Line-Wall Collisions
7 19806 Line-Line Collisions
8 ---- END RESULTS ----
9 depth=2
10 Number of frames = 4000
11 Input file path is: input/mit.in
12 ---- RESULTS ----
13 Elapsed execution time: 1.785175s
14 1262 Line-Wall Collisions
15 19806 Line-Line Collisions
16 ---- END RESULTS ----
17 depth=4
18 Number of frames = 4000
19 Input file path is: input/mit.in
20 ---- RESULTS ----
21 Elapsed execution time: 1.665468s
22 1262 Line-Wall Collisions
23 19806 Line-Line Collisions
24 ---- END RESULTS ----
25 depth=8
26 Number of frames = 4000
27 Input file path is: input/mit.in
28 ---- RESULTS ----
29 Elapsed execution time: 1.874455s
30 1262 Line-Wall Collisions
31 19806 Line-Line Collisions
32 ---- END RESULTS ----
33 depth=16
```

```

34 | Number of frames = 4000
35 | Input file path is: input/mit.in
36 | ---- RESULTS ----
37 | Elapsed execution time: 1.742650s
38 | 1262 Line-Wall Collisions
39 | 19806 Line-Line Collisions
40 | ---- END RESULTS ----
41 | depth=24
42 | Number of frames = 4000
43 | Input file path is: input/mit.in
44 | ---- RESULTS ----
45 | Elapsed execution time: 1.678543s
46 | 1262 Line-Wall Collisions
47 | 19806 Line-Line Collisions
48 | ---- END RESULTS ----
49 | depth=32
50 | Number of frames = 4000
51 | Input file path is: input/mit.in
52 | ---- RESULTS ----
53 | Elapsed execution time: 1.678579s
54 | 1262 Line-Wall Collisions
55 | 19806 Line-Line Collisions
56 | ---- END RESULTS ----

```

depth	执行时间 (s)
1	2.586612
2	1.785175
4	1.665468
8	1.874455
16	1.742650
24	1.678543
32	1.678579

在depth = 4时取得最佳性能

不同depth与MAX_LINES下测试结果

```

1 | lines=1
2 | Number of frames = 4000
3 | Input file path is: input/mit.in
4 | ---- RESULTS ----
5 | Elapsed execution time: 1.999751s
6 | 1262 Line-Wall Collisions
7 | 19806 Line-Line Collisions

```

```
8  ---- END RESULTS ----
9  lines=2
10 Number of frames = 4000
11 Input file path is: input/mit.in
12 ---- RESULTS ----
13 Elapsed execution time: 1.904857s
14 1262 Line-Wall Collisions
15 19806 Line-Line Collisions
16 ---- END RESULTS ----
17 lines=4
18 Number of frames = 4000
19 Input file path is: input/mit.in
20 ---- RESULTS ----
21 Elapsed execution time: 1.816313s
22 1262 Line-Wall Collisions
23 19806 Line-Line Collisions
24 ---- END RESULTS ----
25 lines=8
26 Number of frames = 4000
27 Input file path is: input/mit.in
28 ---- RESULTS ----
29 Elapsed execution time: 1.745250s
30 1262 Line-Wall Collisions
31 19806 Line-Line Collisions
32 ---- END RESULTS ----
33 lines=16
34 Number of frames = 4000
35 Input file path is: input/mit.in
36 ---- RESULTS ----
37 Elapsed execution time: 1.665897s
38 1262 Line-Wall Collisions
39 19806 Line-Line Collisions
40 ---- END RESULTS ----
41 lines=24
42 Number of frames = 4000
43 Input file path is: input/mit.in
44 ---- RESULTS ----
45 Elapsed execution time: 1.646395s
46 1262 Line-Wall Collisions
47 19806 Line-Line Collisions
48 ---- END RESULTS ----
49 lines=32
50 Number of frames = 4000
51 Input file path is: input/mit.in
52 ---- RESULTS ----
53 Elapsed execution time: 1.646755s
54 1262 Line-Wall Collisions
55 19806 Line-Line Collisions
56 ---- END RESULTS ----
```

lines	执行时间 (s)
1	1.999751
2	1.904857
4	1.816313
8	1.745250
16	1.665897
24	1.646395
32	1.646755

在lines = 16时取得最佳性能

五、数据结构重构与鲁棒性细节优化

为了支撑上述算法与并行策略，我在底层数据结构和工程细节上做了大量优化：

1. 全面淘汰链表，引入动态数组：
- 原版使用 IntersectionEventNode 链表，排序需耗费 $O(n^2)$ 的选择排序，且无法配合 Reducer。我将其全部重构为动态数组（ IntersectionEventBuffer 和 CandidatePairBuffer ），支持 $O(1)$ 平摊追加和 $O(n \log n)$ 的 qsort ，为并行归并提供了基础。
2. 平滑的内存扩容策略：
- 在 Quadtree_storeLine 等场景中，采用 1.5 倍或 2 倍动态扩容策略。即使 realloc 失败，也会保留原数组或降级为串行，避免程序直接 Crash，增强了系统鲁棒性。
3. 输入与编译期细节清理：
- 在 fasttime.h 中通过 (void)r; 规避 Release 模式下的未使用变量警告。

◦ 对文件输入 fscanf 的返回值进行严格校验，防止非法格式输入导致未定义行为。

◦ 解除 line.h 对 graphic_stuff.h 的不必要依赖，精简编译链。

◦ 在调用 intersect(l1, l2, timeStep) 时，传入真实的 collisionWorld→timeStep 而非原版的硬编码 0.5 ，使核心逻辑能够动态适应未来的步长调节需求。

六、实验总结

本次优化的核心是：解耦收集与计算、消灭 $O(n^2)$ 瓶颈、以内存换并行时间、无锁化归约。

四叉树 Broad Phase 大幅削减了冗余的相交测试；而动态数组和 Cilk Reducer 的深度结合，使得最为耗时的 Narrow Phase 和数组遍历能够完美吃满多核性能。在彻底杜绝数据竞争并保持物理模拟确定性的前提下，系统的模拟吞吐量得到了质的飞跃。